trails.logos — Product Requirements Document

v0.1.0 · On-chain trail registry and distributed asset storage

Status: Implementation specification
Project: Separate from the native Trails iOS application
Infrastructure: LEZ and Logos Storage
Initial operating model: One funded publisher, operator-run infrastructure

> Publish a complete trail, register it on LEZ, and let another installation discover, download, and revisit it.

1. Purpose

`trails.logos` is the publication and retrieval layer for Trails.

It does not implement camera capture, spatial tracking, or navigation. It stores the information and assets those experiences require, preserves their relationships, and makes published trails discoverable.

The authoritative **registry** lives on LEZ. Trail packages live in Logos Storage.

Required end-to-end outcome

```
Native Trails app exports a trail
→ trails.logos validates the export
→ Logos Storage stores the package
→ LEZ records the publication
→ Another installation discovers the record
→ Downloads and imports the package
→ Replays media and attempts supported retrace alignment
```

Successful publication means both the package and the registry record are verified—not merely that an upload or transaction was submitted.

2. Project boundaries

trails.logos owns

- Trail-package specification.
- Package validation.
- Logos Storage upload and retrieval.
- LEZ registry program.
- Registry deployment and integration.
- Publication lifecycle.
- Public discovery and metadata reads.
- Download delivery.
- Publication receipts.
- Retry and failure handling.

The native Trails application owns

- Capture.
- Local library.
- Exporting compatible packages.
- Importing downloaded packages.
- Media playback.
- Map and spatial rendering.
- Live positioning.
- Alignment verification.
- Retrace sessions.

Out of scope

- Accounts and multi-user onboarding.
- Tokenisation or NFTs.
- Payments for trails.
- Comments, ratings, followers, and likes.
- AI-generated content.
- Turn-by-turn navigation.
- Automatic alignment between independent captures.
- Guaranteed permanent availability.
- A separate database acting as the publication authority.

3. Architecture



A cache is permitted for performance. It must be reconstructable from authoritative registry state and must not create publications independently.

Integration approach

Reuse the working LEZ toolchain, deployment pattern, funded account, and client/runner from the owner's existing setup. Do not introduce unrelated payment logic.

Follow the supplied implementation guide's discipline: distinguish submission from verified state, preserve operation references before ambiguous retries, and record reproducible deployment evidence. [1]

4. What must be stored

Information	Authoritative location
Trail ID and publisher authority	LEZ
Current published revision	LEZ
Current package CID	LEZ
Listed/unlisted state	LEZ
Minimal discovery summary	LEZ
Full trail manifest	Logos Storage
Photos, videos, and voice recordings	Logos Storage
Text notes and moment relationships	Logos Storage
Geographic path observations	Logos Storage
Local spatial path observations	Logos Storage
Coordinate-frame definitions	Logos Storage
Available relocalisation/alignment assets	Logos Storage

Information	Authoritative location
Room geometry and plans, if captured	Logos Storage
Covers and thumbnails	Logos Storage
Temporary uploads and retry state	Local service working storage
Derived discovery cache	Optional, non-authoritative

Private keys must never be included in trail packages, manifests, public records, or API responses.

5. LEZ registry requirements

5.1 Registry configuration

Store:

```
RegistryConfig
  schemaVersion
  administratorAccount
  authorisedPublisherAccount
```

One publisher is sufficient for v0.1.0.

Do not add a multi-user permission system before the publication loop works.

5.2 Current publication record

```
TrailPublication
  trailId
  publisherAccount
  currentRevision
  packageCid
  packageSizeBytes
  packageFormatVersion

  title
  shortDescription?
  recordedAt
  capabilities

  listed
  createdAt
  updatedAt
```

Timestamp provenance must be documented. Do not imply a publisher-supplied timestamp is independently verified chain time.

5.3 Capability summary

```
Capabilities
  hasPhotos
  hasVideos
  hasVoice
  hasText
  hasGeographicPath
  hasLocalSpatialPath
  hasSpatialGeometry
  hasPlan
  hasAlignmentAssets
```

These describe package contents, not guaranteed device compatibility.

`hasAlignmentAssets` must not mean “automatic indoor positioning is guaranteed.”

5.4 Publication revisions

Maintain a registry-accessible revision history:

```
TrailRevision
  trailId
  revision
  packageCid
  packageSizeBytes
  operationId
  publishedAt
```

Do not overwrite an old revision’s CID when publishing an update.

The exact LEZ account layout may differ from this logical schema. The implementation must document how these records map to the deployed program.

6. Registry operations

The following are application operations, not assumed existing SDK methods.

Register trail

```
registerTrail(
  operationId,
  trailId,
  packageCid,
  packageSizeBytes,
  summary,
  capabilities
)
```

Requirements:

- Authorised publisher only.
- Trail ID must not already exist.
- Initial revision is 1.
- Validate field sizes and supported schema.
- Persist enough operation information to resolve retries.

Publish revision

```
publishRevision(
  operationId,
  trailId,
  expectedRevision,
  nextPackageCid,
  nextPackageSizeBytes,
  summary,
  capabilities
)
```

Requirements:

- Publisher authorisation.
- Expected revision must match.
- Increment revision.
- Preserve previous revision reference.
- Reject conflicting repeated operation IDs.

Change listing status

```
setListed(
  operationId,
  trailId,
  listed
)
```

Unlisting affects discovery. It is not a promise to erase distributed content.

Read and discover

Required logical operations:

```
getTrail(trailId)
getRevision(trailId, revision)
listTrails(cursor, limit)
```

The implementation must establish how records are enumerated. Do not assume arbitrary account enumeration is available.

7. Trail-package format

For v0.1.0, use one versioned archive per published revision.

```
package/
  manifest.json

  media/
    photos/
    videos/
    audio/

  paths/
    geographic.json
    local-spatial.json

  spatial/
    geometry/
    plans/
    alignment/

  previews/
    cover.jpg
    thumbnails/
```

Directories and files are optional unless referenced by the manifest.

Package rules

- No absolute device paths.
- No references to files outside the package.
- No required assets available only on the source phone.
- No credentials.
- No hardcoded demonstration content.
- No silent substitution of missing assets.
- Every referenced required file must exist.

The native app must be able to import the package without contacting the publishing phone.

8. Manifest requirements

```
TrailManifest
  schemaVersion
  trailId
  revision

  title
  introduction?
  creatorDisplayName?

  recordedAt
  endedAt?

  place?
  coverAssetId?

  capabilities
  assets[]
  moments[]
  geographicPath?
  spatialSessions[]
  coordinateFrames[]
```

Do not include the outer package CID inside the package itself. The publication receipt supplies that reference after upload.

8.1 Assets

```
Asset
  id
  kind
  relativePath
  mediaType
  byteLength
  sha256
  durationMs?
  width?
  height?
  thumbnailAssetId?
```

Required original files:

- Captured photographs.
- Finalised video with its actual audio track, where recorded.
- Voice recordings.
- Required spatial and alignment files.

Thumbnails alone are insufficient.

8.2 Moments

```
Moment
  id
  startedAt
  endedAt?
  originalOrder
  presentationOrder?

  label?
  text?
  assetIds[]

  geographicContext?
  localSpatialContext?
```

Preserve media-to-moment relationships, not just a folder of files.

8.3 Place

Optional:

```
Place
  localId?
  name?
  locality?
  geographicAnchor?
  floorLabel?
```

A place label does not establish spatial alignment.

9. Path and positioning data

Geographic path

Preserve:

- Timestamp.
- Latitude and longitude.
- Available accuracy.
- Optional altitude and associated accuracy.
- Segment ID.
- Available quality/status information.

Retain gaps and segment boundaries.

Local spatial path

Preserve:

- Timestamp.
- Position.
- Coordinate-frame ID.
- Tracking state.
- Segment ID.
- Orientation when required by the implemented replay/retrace pipeline.

Do not convert local coordinates into fabricated geographic positions.

Coordinate frames

```
CoordinateFrame
  id
  units
  axisConvention
  worldAlignmentConfiguration
  sourceSessionId
  transformRelationships[]
```

Each transform must specify:

- Source frame.
- Destination frame.
- Matrix representation.
- Direction/convention.
- Validation status.

The package must preserve the native application's supported frame semantics.

10. Indoor retrace assets

When available, include:

- Saved relocalisation/world-map data.
- Reference anchors used by the application.
- Reference-area image.
- Marker definition and physical dimensions, if marker-based alignment is used.
- Frame relationships.
- Capture-platform and format compatibility metadata.
- Associated local path.

Required distinction

A package can be:

1. Media-playback compatible.
2. Geographic-retrace compatible.
3. Indoor-alignment capable.
4. Manual-follow-along only.

The receiving app decides whether it can use the supplied assets and whether live alignment succeeds.

Missing indoor alignment assets must not prevent publication of an otherwise valid media trail.

11. Publication flow

```
Receive export
→ Validate package structure
→ Validate manifest and asset references
→ Calculate/check asset digests
→ Upload to Logos Storage
→ Retrieve and verify stored package
→ Submit LEZ registration/update
→ Verify resulting registry state
→ Return publication receipt
```

Receipt

```
PublicationReceipt
  trailId
  revision
  packageCid
  packageSizeBytes
  networkIdentifier
  registryProgramIdentifier
  registryRecordReference
  transactionReference
  verifiedAt
```

Populate actual identifiers returned or established by the implementation.

Success condition

The verified registry record points to the verified package CID.

“Transaction submitted” is not “Published.”

12. Public discovery and download

Discovery response

Return:

- Trail ID.
- Title.
- Short description, if supplied.
- Recording date.
- Revision.
- Capability summary.
- Package size.
- Package CID.

A remote cover can be added later. Do not require downloading an entire large archive merely to display the initial public list.

Download flow

```
Read publication
→ Select revision
→ Retrieve package by CID
→ Validate package
→ Import into new local directory
→ Open trail
```

The receiving app must preserve:

- Original published identity.
- Downloaded revision.
- Publisher reference.
- Source CID.

Local edits must not silently change the published source.

13. Application API

Proposed `trails.logos` endpoints:

```
POST /publications
GET  /publications/{operationId}

GET  /trails
GET  /trails/{trailId}
GET  /trails/{trailId}/revisions/{revision}
```

```
GET  /trails/{trailId}/download

POST /trails/{trailId}/revisions
POST /trails/{trailId}/listing
```

These are service endpoints, not Logos APIs.

Requirements:

- Authenticate write operations.
- Bound upload and response sizes.
- Support idempotent publication requests.
- Expose actionable errors.
- Keep raw node administration interfaces private.

14. Publisher key handling

Use one dedicated development publisher funded for this deployment.

Preferred configuration:

- Server-side environment variable or ignored local configuration.
- No real-value account.
- No reuse of an unrelated administrator key.

A fixed development key is acceptable for the private prototype. If temporarily hardcoded, keep it in an ignored, deployment-local server configuration—not in the iOS app, public repository, package, or browser bundle.

Anyone with that key can exercise its publisher permissions. Replace it before distributing the deployment beyond the controlled demonstration.

No wallet onboarding or general sponsorship system is required for v0.1.0.

15. Failure and retry behaviour

Failure	Required behaviour
Invalid export	Reject before upload
Missing asset	Identify the missing reference
Storage upload failure	Retain local export; retry
Stored package fails verification	Do not register
Registry submission fails	Retain CID and operation ID
Submission outcome ambiguous	Read authoritative state before retry

Failure	Required behaviour
Discovery cache unavailable	Report failure or use supported direct reads
Package unavailable	Preserve registry identity; explain retrieval failure
Unsupported alignment format	Allow compatible playback; disable live indoor retrace

Never silently fall back to an off-chain publication record.

16. Public-data confirmation

Before publication, show:

> “This trail includes recorded media and may include location and spatial data. Others may download and retain a copy.”

Require confirmation.

Unlisting copy:

> “This removes the trail from public listings. Existing downloaded copies are not removed.”

Do not promise permanent storage or universal deletion.

17. Acceptance criteria

Registry

- Register a real package CID.
- Read it back from LEZ.
- Reject an unauthorised update.
- Publish revision 2 without destroying revision 1.
- Retry an operation without duplicate publication.
- Unlist and relist.
- Discover publications from authoritative state.

Storage

- Upload a complete real trail.
- Retrieve it through the configured network path.
- Validate its contents.
- Play actual photo, video, and voice assets.
- Preserve paths, moments, and coordinate frames.

Receiving installation

- Start with no local copy.
- Discover a publication.
- Download and import.
- Open all supported content.
- Attempt alignment only when applicable.
- Reopen downloaded content offline.

Evidence

Deliver:

- Registry program and deployment identifiers.
- Verified transaction/state references.
- Example published CID.
- Package schema.
- Reproducible publish/download commands.
- Explicit working, incomplete, and failed checks.

18. Definition of done

`trails.logos` v0.1.0 is complete when:

> A real trail can be published to Logos Storage, registered authoritatively on LEZ, discovered by another installation, downloaded intact, and opened with the media and positioning context needed for its supported experiences.

The registry establishes what was published. Storage carries the experience. The native app brings it back to life.

⁂

1. Dhan_v0.1.0_Dev_Agent_Implementation_Guide.md